CO1_AT2_Simulation

1.Design a Non-Deterministic Finite Automaton (NBA) for an Automatic Door System based on sensor inputs.An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: Closed, Opening, Open, and Closing. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the Closed state. When a person is detected (P), the system may non-deterministically transition from Closed to Opening. Once opened, the system moves to the Open state. If no person is detected (N), the door transitions to Closing and eventually returns to the Closed state. Non-determinism occurs when the system may remain in the Open state even if a person is detected or may start closing if no detection occurs.

1. Define the NBA

M = (Q,Σ,δ,q0,F)

M = (Q, \Sigma, \delta, q_0, F)
M=(Q,Σ,δ,q0 ,F)

- **Q** = {C, OP, O, CL}
  - C = Closed
  - OP = Opening
  - O = Open
  - CL = Closing
- **Σ** = {P, N}
  - P = Person detected
  - N = No person detected
- $\mathbf{q_0}$ = C (Initial State)
- **F** = {O} (Accept/Final State)

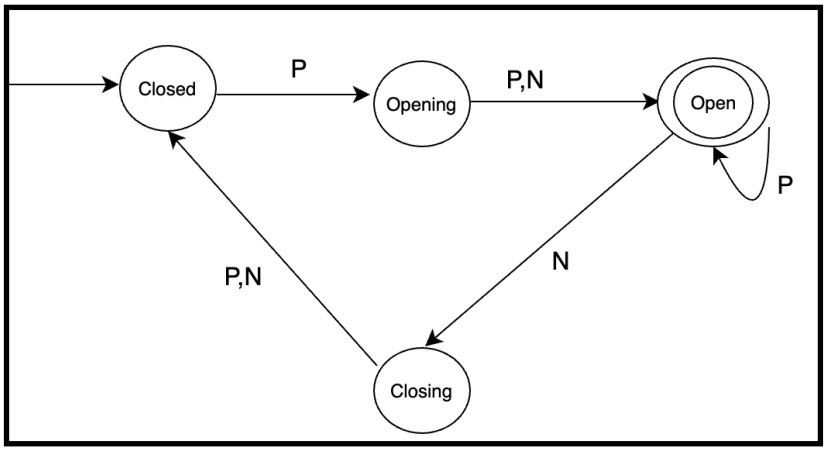
2. Transition Table:

Initial State: → C

Final State: *O

Current State	Input P	Input N
→ C	{OP}	{C}
OP	{O}	{O}
*O	{O}	{O, CL}
CL	{C}	{C}

3. State Transition Diagram



The door starts in the **Closed (C)** state.
On input **P (Person detected)**, it moves to **Opening (OP)**.
From **Opening**, it reaches the **Open (O)** state.
In the **Open** state:

- On **P**, it remains **Open**.
- On **N**, it can **either remain Open or move to Closing (CL)**. This is the **non-deterministic transition**.

From **Closing**, the door returns to **Closed (C)**.

2.Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

1. DFA Components:

$$M=(Q,\Sigma,\delta,q0\ ,F)$$

Q = {R, G, Y}

- R = Red
- G = Green
- Y = Yellow

Σ = {T}

- T = Timer expires

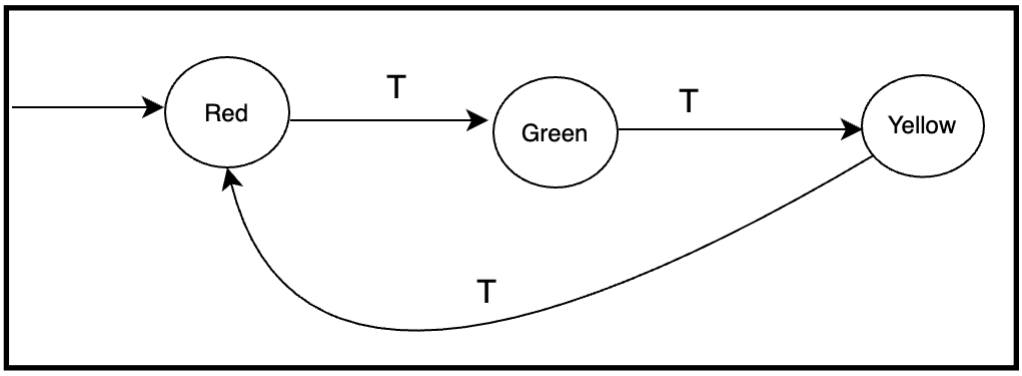
$\mathbf{q_0}$ = R (Initial State)

F = {R, G, Y} (All states are accepting since the system runs continuously.)

2. Transition Table

Current State	Input (T)	Next State
R	T	G
G	T	Y
Y	T	R

3. State Diagram



- **Red:** 30 seconds
- **Green:** 40 seconds
- **Yellow:** 5 seconds

After the timer expires:

- Red → Green
- Green → Yellow
- Yellow → Red

This cycle repeats continuously.

The automaton never stops. After reaching the Yellow state, it automatically returns to Red, creating an infinite cycle:

Red → Green → Yellow → Red → Green → Yellow → ...

4.Extension for Pedestrian Crossing

Add a **Pedestrian Walk (P)** state.

When a pedestrian button is pressed:

- Green → Yellow → Red → Pedestrian Walk
- After the pedestrian timer ends:
- Pedestrian Walk → Green

5.Extension for Emergency Vehicle Override

Add an **Emergency (E)** input.

If an emergency vehicle is detected:

- Any State → Emergency Green
- After the emergency passes:
- Emergency Green → Red

- Then normal cycling resumes.

3.Design DFA using simulator to accept the string the end with ab over set {a,b) W= aaabab

To design a **DFA that accepts all strings ending with "ab"** over the alphabet:

Σ={a,b}

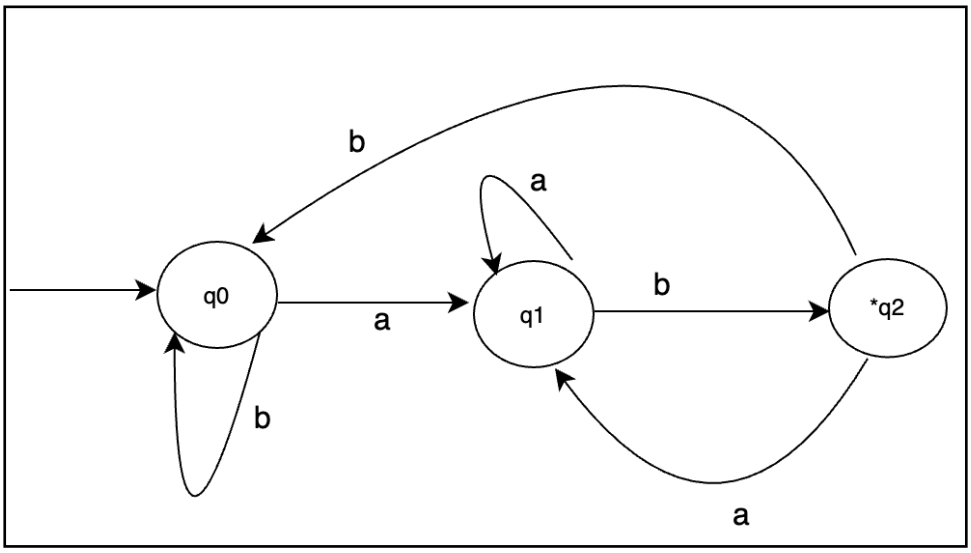
States

- **q0** → Initial state (no match yet)
- **q1** → Last symbol is **a**
- **q2** → Last two symbols are **ab** (**Final state**)

Transition Table

Current State	a	b
→ q0	q1	q0
q1	q1	q2
*q2	q1	q0

DFA Diagram



States: q0, q1, q2
Initial state: q0
Final state: q2

Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings "c", "bc", and "bcaaa" over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string "c" should be accepted directly from the start state, while "bc" should pass through states representing "b" and then "c". Similarly, "bcaaa" should be processed through a sequence of states that track each additional "a" after "bc". Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.

Step 1: Draw the states

→ q0 q1 q2 q3 q4 q5 qd

q0 = Start state
q2 = Accepts **c**
q3 = Accepts **bc**
q4 = After reading **bca**
q5 = After reading **bcaa**
q6 = Accepts **bcaaa**
qd = Dead state

Actually you'll need **8 states**:

→ q0 q1 *q2 *q3 q4 q5 *q6 qd

Step 2: Draw the main path

String **c :** → **q0 ----c----> *q2**

String **bc:** → **q0 ----b----> q1 ----c----> *q3**

String **bcaaa: *q3 ----a----> q4 ----a----> q5 ----a----> *q6**

Final diagram:

+--------> ((q2)) c

→ (q0) ----b----> (q1) ----c----> ((q3)) ----a----> (q4) ----a----> (q5) ----a----> ((q6))

Now add the **dead state**.

Dead state

Draw one more state below. (qd)

Transition Table

State	a	b	c
→ q0	qd	q1	q2
q1	qd	qd	q3
*q2	qd	qd	qd
*q3	q4	qd	qd
q4	q5	qd	qd
q5	q6	qd	qd
*q6	qd	qd	qd
qd	qd	qd	qd

Accepting states

Draw **double circles** around:

- **q2** (accepts c)
- **q3** (accepts bc)
- **q6** (accepts bcaaa)

5. Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either “a” or “b”. The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either “a” or “b”. After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of “a” and “b”, thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

DFA

States

- **q0** = Start state
- **q1** = Accepting state
- **qd** = Dead state (optional, for simulator)

Transition Table

Current State	a	b
→ q0	q1	q1
*q1	q1	q1
qd	qd	qd

DFA Diagram